

Revision 01 • What Is Inference Engineering

Interactive revision of [lesson 01](#): each question as asked, how I reasoned it out (my words, written in full), where I slipped, and the one-line takeaway.

Q1 • Can one user get 300 tokens/s? SOLID

Covers: 01, 16

QUESTION

We serve Llama 3 8B in BF16 on one H100, with only one user online. The PM wants to advertise "300 tokens/s streaming". Can we promise it?

How I solved it

1. The model has about 8 billion parameters, and BF16 stores each in 2 bytes, so the weights are about **16 GB**.
2. An H100's memory bandwidth is about **3.35 TB/s**.
3. Every decode step reads all the weights once: $16 \times 10^9 \text{ bytes} \div 3.35 \times 10^{12} \text{ bytes/s} = (16 \div 3.35) \times 10^{-3} \text{ s} \approx \textbf{4.8 ms}.$
4. In one second that's $1 \div 4.8 \text{ ms} \approx 200$ steps, one token each: about **200 tokens/s at most**.
5. 300 is above that ceiling, so the answer is **no**.

What to add next time

- Name the limiting resource out loud: **memory bandwidth**, not compute. The math units are mostly idle.
- 200 tokens/s is a **ceiling**, not the expected speed. Real batch-1 runs land 3–4× slower (lesson 16), and the ceiling drops as the KV cache grows.
- To reach 300, read fewer bytes: 8-bit weights are 8 GB $\rightarrow \sim 2.4 \text{ ms} \rightarrow$ a ceiling of ~ 415 tokens/s.

Q2 • 64 users at once: step time and speed PARTIAL

Covers: 01, 16

QUESTION

Same GPU and model, but 64 users are chatting at once, each with about 4,096 tokens of context in their KV cache (128 KB per token). How long is one decode step, what is the total tokens/s, and what does each user get?

How I solved it (with hints)

1. I couldn't recall the formula, so I went back to the principle from Q1: **time per step $\approx$ bytes read in that step $\div$ bandwidth**. The only real question is which bytes get read.
2. **Weights**: batched, so the 16 GB is read **once per step for everyone**, not once per user.
3. **KV cache**: each user's new token attends to that user's own past tokens, so every user's whole cache is read: $4,096 \times 128 \text{ KB} \approx 0.5 \text{ GB}$ each, $\times 64 \approx$ **32 GB** (about 34 GB in decimal units).
4. **Total read per step**: $16 + 32 \approx 48 \text{ GB}$, $\div 3.35 \text{ TB/s} \approx$ **14 ms** (the lessons get 15 ms with exact sizes).
5. **Per user**: $14 \text{ ms} = 0.014 \text{ s}$, so $1 \div 0.014 \approx$ **71 tokens/s**.
6. **Total**: each step gives all 64 users a token, so total = batch $\times$ (1 $\div$ step time) = batch $\div$ step time: $64 \div 0.014 \text{ s} = 71 \times 64 \approx$ **4,500 tokens/s**.
7. **Versus batch 1**: the GPU makes $4,500 \div 200 \approx$ **22 $\times$ more**, while each user slows from 200 to 71, only **$\sim 2.8\times$ slower**.

The 16 GB weight read is shared by everyone; only the KV cache is per user. The price of batching is KV memory (32 GB here), growing with batch $\times$ context.

RE-DRILL

Per-request KV = tokens in the conversation $\times$ 128 KB. Always sanity-check ms $\rightarrow$ tokens/s against batch 1 (4.8 ms $\rightarrow$ 200).

Q3 · Cost per million tokens **PARTIAL**

Covers: 01

QUESTION

The H100 rents for \$3/hour. What does 1 million output tokens cost at ~ 200 tokens/s (one user), and at $\sim 4,500$ tokens/s (64 users batched)?

Step-by-step answer. Cost = price per hour $\times$ hours of GPU needed.

Step	One user (200 tok/s)	64 users (4,500 tok/s)
Time for 1M tokens	$1,000,000 \div 200 =$ 5,000 s	$1,000,000 \div 4,500 \approx$ 222 s
Convert to hours	$5,000 \div 3,600 \approx$ 1.39 h	$222 \div 3,600 \approx$ 0.06 h
Cost = \$3/h $\times$ hours	\$4.17	\$0.18

Ratio: $4.17 \div 0.18 \approx$ **22 $\times$ cheaper**, the same factor as the throughput gain, because the GPU's hourly price doesn't change.

RE-DRILL

Write units next to every number (s, h, \$/h) so an upside-down formula shows itself. Sanity check: "about an hour, so about \$3?"

Q4 · Why is prefill of 500 tokens only ~2× one decode token? **PARTIAL**

Covers: 01, 15

QUESTION

Prefill pushes a 500-token prompt through in ~8 ms; one decode token takes ~4.8 ms. Why so close? Which resource limits each phase?

My explanation (concept, correct). In prefill all the prompt tokens are available up front, so nothing waits and the work runs in parallel: the compute units stay busy, so prefill is compute-intensive. In decode only one token at a time flows through all the weights, so the GPU spends most of its time waiting for data: memory-intensive.

Gap: I couldn't show *with numbers* that decode is memory-bound.

Step-by-step proof: compare math time with byte time; the bigger one sets the speed.

- Facts:** each parameter does 2 FLOPs per token (multiply + add). H100 $\approx$ 989 TFLOPS and 3.35 TB/s.
- Decode math:** $2 \times 8\text{B} = 16 \times 10^9$ FLOPs. GPU: 989 TFLOPS = 989×10^{12} FLOP/s $\approx 1,000 \times 10^{12} = 10^{15}$ FLOP/s. Divide: $16 \times 10^9 \div 10^{15} = 16 \times 10^{-6} \text{ s} = \mathbf{16 \mu\text{s}}$.
- Decode bytes:** 16×10^9 bytes $\div 3.35 \times 10^{12}$ bytes/s = $(16 \div 3.35) \times 10^{-3} \text{ s} = \mathbf{4.8 \text{ ms}}$ ($3.35 \times 5 \approx 16.75$, so $16 \div 3.35 \approx 4.8$).
- $4.8 \text{ ms} \div 16 \mu\text{s} \approx \mathbf{300\times}$: decode spends ~300× more time waiting for bytes than doing math. Math units idle >99%: **memory-bound**.
- Ridge point:** 989×10^{12} FLOP/s $\div 3.35 \times 10^{12}$ B/s; tera and seconds cancel $\rightarrow 989 \div 3.35 \approx \mathbf{295 \text{ FLOP/byte}}$ ($3.35 \times 300 \approx 1,000$). Batch-1 decode does ~1.
- Prefill 500 tokens:** weights still read once (4.8 ms), but math = $500 \times 16 \mu\text{s} = \mathbf{8 \text{ ms}}$. Math > bytes: **compute-bound**, $\approx 8 \text{ ms}$.

RE-DRILL

Powers-of-ten division: giga $\div$ tera = 10^{-3} , so $16 \text{ G} \div 1,000 \text{ T} = 16 \mu\text{s}$. Say "max(math, bytes)" out loud.

Exercise · Napkin calculator in code **PARTIAL**

Covers: 01, 16

QUESTION

Write `decode_step(params_b, bytes_per_param, batch, ctx_tokens, ...)` returning step time (ms), total tokens/s, per-user tokens/s, and whether it's memory- or compute-bound.

Gap: I knew the hand math but couldn't structure it as code: where to start, in what order.

The structure to reuse (same chain of thought as by hand):

1. **Read the inputs:** what each means physically (params in billions, bytes per param = precision, batch = users per step, ctx = tokens in each user's cache).
2. **Normalize** to plain units: $\text{params} \times 10^9$, $\text{TB/s} \times 10^{12}$, $\text{TFLOPS} \times 10^{12}$.
3. **Bytes per step:** weights once + batch $\times$ ctx $\times$ KV per token ($\text{KB} \times 1024$) $\rightarrow \div$ bandwidth = byte time.
4. **Math per step:** $2 \times \text{params} \times \text{batch FLOPs} \rightarrow \div \text{FLOPs/s} = \text{math time}$.
5. **Bottleneck:** $\text{step} = \max(\text{byte time}, \text{math time})$; bound = whichever was bigger.
6. **Speeds:** per user = $1 \div \text{step}$; total = per user $\times$ batch.

```
def decode_step(params_b, bytes_per_param, batch, ctx_tokens,
                kv_kb_per_token=128, bw_tbs=3.35, tflops=989):
    # Step 1 · Normalize to plain units
    params = params_b * 10**9          # parameters
    bw = bw_tbs * 10**12               # bytes per second
    flops = tflops * 10**12           # FLOPs per second

    # Step 2 · Bytes: weights once (shared) + every user's whole cache
    weight_bytes = params * bytes_per_param
    kv_bytes = batch * ctx_tokens * kv_kb_per_token * 1024
    t_bytes = (weight_bytes + kv_bytes) / bw      # seconds

    # Step 3 · Math: 2 FLOPs per param per token, one token per user
    flops_needed = 2 * params * batch
    t_math = flops_needed / flops                # seconds

    # Step 4 · Bottleneck: the slower one sets the pace
    step_s = max(t_bytes, t_math)
    bound = "memory" if t_bytes >= t_math else "compute"

    # Step 5 · Speeds: each step gives every user one token
    per_user_tok_s = 1 / step_s
    total_tok_s = per_user_tok_s * batch
    return step_s * 1000, total_tok_s, per_user_tok_s, bound

decode_step(8, 2, 1, 0)      # (4.78 ms, 209 tok/s, 209, 'memory')
decode_step(8, 2, 64, 4096) # (15.0 ms, 4,257 tok/s, 66.5, 'memory')
```

Q5 · At what batch does decode become compute-bound?

SOLID

Covers: 01

QUESTION

With zero context, raise the batch until `decode_step` says 'compute'. Where does it flip, and which number does that match?

My answer: around 300, the ridge point. **Checked with the code:** batch 295 → memory, 296 → compute; ridge = $989 \div 3.35 \approx 295$.

Why batch = ridge: in BF16 one step does $2 \times P \times B$ FLOPs and reads $2 \times P$ bytes of weights, so FLOPs per byte = **B**. Decode turns compute-bound once B passes the ridge.

Caveat: with real context every user adds KV bytes too, so the flip comes much later or never: batch 296 at 4k context is still memory-bound.